Navegación: inicio y registro

Hoy la navegación queda en dos pantallas: inicio y registro. La de registro solo muestra un texto. Mañana el formulario se agrega dentro de esa misma pantalla, sin cambiar la ruta.

1. Dependencias

En `app/build.gradle.kts`, dentro de `dependencies`, agrega esto y sincroniza Gradle:

```
implementation("androidx.navigation:navigation-compose:2.7.7")
implementation("androidx.lifecycle:lifecycle-viewmodel-compose:2.8.0-rc02")
implementation("org.jetbrains.kotlinx:kotlinx-coroutines-android:1.7.3")
```

Estas líneas descargan tres librerías que el proyecto todavía no tiene.

- `navigation-compose` trae `NavHost`, `rememberNavController` y `composable`. Con eso la app puede cambiar de pantalla.
- `lifecycle-viewmodel-compose` trae `viewModel()`. Así `MainActivity` y la preview obtienen el mismo `MainViewModel` sin crearlo a mano.
- `kotlinx-coroutines-android` permite emitir el evento de navegación en segundo plano y entregarlo a la pantalla cuando esté lista.

Sin estas librerías, el código de los puntos siguientes no compila. Después de pegarlas hay que sincronizar Gradle para que Android Studio las baje.

2. La ruta

Crea el paquete `cl.duoc.myapplication.navigation` y el archivo `Screen.kt`. Cada pantalla tiene un nombre fijo:

```
package cl.duoc.myapplication.navigation

sealed class Screen(val route: String) {
    data object Home : Screen("home_page")
    data object Register : Screen("register_page")
}
```

`Screen` es la lista cerrada de pantallas. Cada una guarda un texto, la ruta, que identifica esa pantalla dentro de la navegación.

- `Home` vale `"home_page"` y es el inicio.
- `Register` vale `"register_page"` y es el registro.

`sealed class` obliga a usar solo estas opciones. Si alguien escribe una ruta que no existe, el compilador lo marca. El `NavHost` del punto 7 usa `Screen.Home.route` y `Screen.Register.route` para saber qué composable mostrar.

3. El evento

En el mismo paquete, `NavigationEvent.kt`. La pantalla no cambia de vista: avisa que quiere ir a otra.

```
package cl.duoc.myapplication.navigation

sealed class NavigationEvent {
    data class NavigateTo(val route: Screen) : NavigationEvent()
    data object PopBackStack : NavigationEvent()
}
```

Un evento es un aviso. La pantalla lo envía y `MainActivity` decide qué hacer.

- `NavigateTo` lleva la pantalla de destino. Para ir al registro, el aviso es `NavigateTo(Screen.Register)`.
- `PopBackStack` significa volver a la pantalla anterior. Hoy lo usa el botón atrás del teléfono; mañana puede usarlo un botón Volver.

La pantalla de inicio no llama a `navController.navigate`. Solo crea este aviso y se lo pasa al `ViewModel`.

4. El ViewModel

Crea el paquete `cl.duoc.myapplication.viewmodels` y `MainViewModel.kt`:

```
package cl.duoc.myapplication.viewmodels

import androidx.lifecycle.ViewModel
import cl.duoc.myapplication.navigation.NavigationEvent
import cl.duoc.myapplication.navigation.Screen
import kotlinx.coroutines.CoroutineScope
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.flow.MutableSharedFlow
import kotlinx.coroutines.flow.asSharedFlow
import kotlinx.coroutines.launch

class MainViewModel : ViewModel() {
    private val _navigationEvents = MutableSharedFlow<NavigationEvent>()
    val navigationEvents = _navigationEvents.asSharedFlow()

    fun navigateTo(screen: Screen) {
        CoroutineScope(Dispatchers.Main).launch {
            _navigationEvents.emit(NavigationEvent.NavigateTo(screen))
        }
    }
}
```

El `ViewModel` es el puente entre el botón y `MainActivity`. Sigue vivo aunque la pantalla se redibuje.

- `_navigationEvents` es el canal privado donde se publican los avisos. El guion bajo indica que nadie de afuera escribe ahí.
- `navigationEvents` es la versión pública, de solo lectura. `MainActivity` se suscribe a ella.

- `navigateTo` arma un `NavigateTo` con la pantalla recibida y lo publica en el canal. `HomeScreen` llama a esta función cuando pulsan Registrarse.

`emit` es una función suspendida, por eso va dentro de `launch`. El aviso sale en el hilo principal, que es donde Compose actualiza la interfaz.

5. La pantalla de registro

Archivo `ui/RegisterScreen.kt`. Hoy solo un texto:

```
package cl.duoc.myapplication.ui

import androidx.compose.material3.Text
import androidx.compose.runtime.Composable

@Composable
fun RegisterScreen() {
    Text("Pantalla de registro")
}
```

`RegisterScreen` es la segunda pantalla. `@Composable` la marca como función de interfaz: Compose la dibuja cuando el `NavHost` entra a la ruta `"register_page"`.

Hoy el cuerpo solo muestra el texto “Pantalla de registro”. Sirve para comprobar que la navegación funciona antes de armar el formulario. Mañana los campos y el botón se agregan dentro de esta misma función.

6. El botón Registrarse

En `HomeScreen`, recibe el `ViewModel` y el botón emite la ruta:

```
@Composable
fun HomeScreen(viewModel: MainViewModel) {
```

`HomeScreen` ahora recibe el `ViewModel` por parámetro. El botón, que está dentro de esta función, puede llamar a `viewModel.navigateTo(...)`. El `ViewModel` no se crea aquí: lo crea `MainActivity` y lo entrega al abrir el inicio.

```
Button(
    onClick = { viewModel.navigateTo(Screen.Register) }
) {
    Text("Registrarse")
}
```

`onClick` se ejecuta al pulsar el botón. En ese momento llama a `navigateTo` con `Screen.Register`, así que el aviso que sale es “quiero ir a registro”. El botón no abre la pantalla. Solo publica el evento.

Imports en ese archivo:

```
import cl.duoc.myapplication.navigation.Screen
import cl.duoc.myapplication.viewmodels.MainViewModel
```

Estos imports permiten usar `Screen` y `MainViewModel` en `HomeScreen`. Sin ellos, Kotlin no sabe en qué paquete están esas clases.

La preview queda así, porque ahora `HomeScreen` pide el `ViewModel`:

```
@Preview(showBackground = true)
@Composable
fun HomeScreenPreview() {
    MyApplicationTheme {
        HomeScreen(viewModel = viewModel())
    }
}
```

La preview dibuja `HomeScreen` en Android Studio sin correr la app. Como la función exige un `MainViewModel`, `viewModel()` le entrega uno para que el diseño se pueda ver. `MyApplicationTheme` envuelve la preview para que los botones hereden los colores del tema.

```
import androidx.lifecycle.viewmodel.compose.viewModel
```

Este import trae la función `viewModel()` que usa la preview.

7. MainActivity

`MainActivity` escucha el evento y muestra la pantalla que corresponde. Reemplaza el `setContent` por esto:

```
setContent {
    MyApplicationTheme {
        val viewModel: MainViewModel = viewModel()
        val navController = rememberNavController()

        LaunchedEffect(Unit) {
            viewModel.navigationEvents.collectLatest { event ->
                when (event) {
                    is NavigationEvent.NavigateTo ->
                        navController.navigate(event.route.route)
                    NavigationEvent.PopBackStack ->
                        navController.popBackStack()
                }
            }
        }

        NavHost(
            navController = navController,
            startDestination = Screen.Home.route
        ) {
            composable(Screen.Home.route) {
                HomeScreen(viewModel = viewModel)
            }
            composable(Screen.Register.route) {
                RegisterScreen()
            }
        }
    }
}
```

Este bloque es el que realmente cambia de pantalla.

- `viewModel()` crea el `MainViewModel` y se lo pasa a `HomeScreen`. Es el mismo objeto que escucha los clics.
- `rememberNavController()` crea el controlador. Guarda la pila de pantallas: inicio queda abajo y registro se apila encima.
- `LaunchedEffect` se suscribe a `navigationEvents` mientras la actividad está en pantalla. Cada aviso que llega entra al `when`.
- Si el aviso es `NavigateTo`, `navigate` abre la ruta que trae el evento. Para registro, esa ruta es `"register_page"`.
- Si el aviso es `PopBackStack`, saca la pantalla de arriba y reaparece la anterior.
- `NavHost` es el contenedor. `startDestination` dice que al abrir la app se muestra `home_page`.
- Cada `composable` asocia una ruta con una función. `"home_page"` dibuja `HomeScreen` y `"register_page"` dibuja `RegisterScreen`.

Imports nuevos:

```
import androidx.compose.runtime.LaunchedEffect
import androidx.lifecycle.viewmodel.compose.viewModel
import androidx.navigation.compose.NavHost
import androidx.navigation.compose.composable
import androidx.navigation.compose.rememberNavController
import cl.duoc.myapplication.navigation.NavigationEvent
import cl.duoc.myapplication.navigation.Screen
import cl.duoc.myapplication.ui.RegisterScreen
import cl.duoc.myapplication.viewmodels.MainViewModel
import kotlinx.coroutines.flow.collectLatest
```

Cada import corresponde a un nombre usado en `setContent`: el efecto, el `ViewModel`, el `NavHost`, las rutas, las pantallas y `collectLatest`, que es la función que recibe los eventos uno por uno.

Qué ocurre al pulsar Registrarse

1. El botón llama a `navigateTo(Screen.Register)`.
2. El `ViewModel` emite el evento.
3. `MainActivity` ejecuta `navigate("register_page")`.
4. Aparece el texto.
5. El botón atrás del teléfono vuelve al inicio.

Mañana

El formulario (campos y botón) se escribe dentro de `RegisterScreen`. `Screen.Register` y el `NavHost` se quedan igual.